DS HW 6

1. Simulate the undo and redo operation in an MS-Word package using a doubly linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
    char action[50];
    struct Node *prev;
    struct Node *next;
};

struct Node *current = NULL;

void addAction(char action[]) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    strcpy(newNode->action, action);
    newNode->prev = current;
    newNode->next = NULL;

    if (current != NULL)
        current->next = newNode;

    current = newNode;
}

void undo() {
    if (current == NULL)
        printf("Nothing to undo.\n");
    else {
        printf("Undo: %s\n", current->action);
        current = current->prev;
    }
}

void redo() {
    if (current == NULL)
        printf("Nothing to redo.\n");
    else {
        current = current->next;
        printf("Redo: %s\n", current->action);
    }
}

int main() {
    addAction("Type Hello");
    addAction("Type World");
    addAction("Delete World");

    printf("Undo/Redo operations:\n");

    undo();
    undo();

    redo();

    return 0;
}
```

Output:

```
Undo/Redo operations:
Undo: Delete World
```

```
Undo: Type World
Redo: Type World
```

2. Given two sorted lists L1 and L2, write a procedure to compute L1 ∩ L2 and L1 ∪ L2 using basic list operations.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void insert(struct Node **head, int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));
    struct Node *temp;

    newNode->data = data;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

struct Node *intersection(struct Node *a, struct Node *b) {
    struct Node *result = NULL;

    while (a != NULL && b != NULL) {
        if (a->data == b->data) {
            insert(&result, a->data);
            a = a->next;
            b = b->next;
        }
        else if (a->data < b->data)
            a = a->next;
        else
            b = b->next;
    }

    return result;
}

struct Node *unionList(struct Node *a, struct Node *b) {
    struct Node *result = NULL;

    while (a != NULL && b != NULL) {
        if (a->data < b->data) {
            insert(&result, a->data);
            a = a->next;
        }
        else if (a->data > b->data) {
            insert(&result, b->data);
            b = b->next;
        }
        else {
            insert(&result, a->data);
            a = a->next;
```

```
            b = b->next;
        }
    }

    while (a != NULL) {
        insert(&result, a->data);
        a = a->next;
    }

    while (b != NULL) {
        insert(&result, b->data);
        b = b->next;
    }

    return result;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct Node *L1 = NULL, *L2 = NULL;
    struct Node *inter, *uni;

    insert(&L1, 1);
    insert(&L1, 3);
    insert(&L1, 5);
    insert(&L1, 7);

    insert(&L2, 2);
    insert(&L2, 3);
    insert(&L2, 5);
    insert(&L2, 8);

    inter = intersection(L1, L2);
    uni = unionList(L1, L2);

    printf("L1: ");
    display(L1);

    printf("L2: ");
    display(L2);

    printf("L1 intersection L2: ");
    display(inter);

    printf("L1 union L2: ");
    display(uni);

    return 0;
}
```

Output:

```
L1: 1 3 5 7
L2: 2 3 5 8
L1 intersection L2: 3 5
L1 union L2: 1 2 3 5 7 8
```

3. Swap two elements by adjusting only the pointers (and not the data) using a doubly linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

void insert(struct Node **head, int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));
    struct Node *temp;

    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

void swapNodes(struct Node **head, struct Node *a,
               struct Node *b) {
    struct Node *ap = a->prev, *an = a->next;
    struct Node *bp = b->prev, *bn = b->next;

    if (a->next == b) {
        if (ap) ap->next = b;
        else *head = b;

        if (bn) bn->prev = a;

        b->prev = ap;
        b->next = a;
        a->prev = b;
        a->next = bn;
    }
    else if (b->next == a) {
        swapNodes(head, b, a);
    }
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct Node *head = NULL, *temp, *a, *b;

    insert(&head, 10);
    insert(&head, 20);
    insert(&head, 30);
    insert(&head, 40);

    a = head->next;          /* 20 */
    b = head->next->next;    /* 30 */

    printf("Before swapping: ");
    display(head);
```

```
    swapNodes(&head, a, b);

    printf("After swapping: ");
    display(head);

    return 0;
}
```

Output:

```
Before swapping: 10 20 30 40
After swapping: 10 30 20 40
```

4. Write an array implementation of a self-adjusting list. Except for the find operation, all operations are performed at the front. When an element is accessed by find, move it to the front without changing the relative order of the other elements.

Program:

```
#include <stdio.h>

#define MAX 20

int list[MAX];
int n = 0;

void insertFront(int value) {
    int i;

    for (i = n; i > 0; i--)
        list[i] = list[i - 1];

    list[0] = value;
    n++;
}

int find(int value) {
    int i, temp;

    for (i = 0; i < n; i++) {
        if (list[i] == value) {
            temp = list[i];

            for (; i > 0; i--)
                list[i] = list[i - 1];

            list[0] = temp;
            return 1;
        }
    }

    return 0;
}

void display() {
    int i;

    for (i = 0; i < n; i++)
        printf("%d ", list[i]);

    printf("\n");
}

int main() {
    insertFront(10);
    insertFront(20);
    insertFront(30);
    insertFront(40);
```

```
    printf("Initial list: ");
    display();

    find(20);

    printf("After finding 20: ");
    display();

    return 0;
}
```

Output:

```
Initial list: 40 30 20 10
After finding 20: 20 40 30 10
```

5. Write a C program to reverse a given doubly linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

void insert(struct Node **head, int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));
    struct Node *temp;

    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

void reverse(struct Node **head) {
    struct Node *current = *head;
    struct Node *temp = NULL;

    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }

    if (temp != NULL)
        *head = temp->prev;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
```

```
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct Node *head = NULL;

    insert(&head, 10);
    insert(&head, 20);
    insert(&head, 30);
    insert(&head, 40);

    printf("Original list: ");
    display(head);

    reverse(&head);

    printf("Reversed list: ");
    display(head);

    return 0;
}
```

Output:

```
Original list: 10 20 30 40
Reversed list: 40 30 20 10
```

6. Write a C program to delete the middle element of a doubly linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

void insert(struct Node **head, int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));
    struct Node *temp;

    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

void deleteMiddle(struct Node **head) {
    struct Node *slow = *head, *fast = *head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }

    if (slow->prev != NULL)
        slow->prev->next = slow->next;
    else
        *head = slow->next;

    if (slow->next != NULL)
        slow->next->prev = slow->prev;

    free(slow);
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct Node *head = NULL;

    insert(&head, 10);
    insert(&head, 20);
    insert(&head, 30);
    insert(&head, 40);
    insert(&head, 50);
```

```
    printf("Before deletion: ");
    display(head);

    deleteMiddle(&head);

    printf("After deleting middle: ");
    display(head);

    return 0;
}
```

Output:

```
Before deletion: 10 20 30 40 50
After deleting middle: 10 20 40 50
```

7. Remove duplicate elements from (i) a sorted doubly linked list and (ii) an unsorted doubly linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

void insert(struct Node **head, int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));
    struct Node *temp;

    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    temp = *head;
    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;
}

void removeSortedDuplicates(struct Node *head) {
    struct Node *temp = head, *dup;

    while (temp != NULL && temp->next != NULL) {
        if (temp->data == temp->next->data) {
            dup = temp->next;
            temp->next = dup->next;

            if (dup->next != NULL)
                dup->next->prev = temp;

            free(dup);
        } else {
            temp = temp->next;
        }
    }
}
```

```
void removeUnsortedDuplicates(struct Node *head) {
    struct Node *current, *runner, *dup;

    for (current = head; current != NULL;
         current = current->next) {

        runner = current->next;

        while (runner != NULL) {
            if (runner->data == current->data) {
                dup = runner;
                runner = runner->next;

                if (dup->prev != NULL)
                    dup->prev->next = dup->next;

                if (dup->next != NULL)
                    dup->next->prev = dup->prev;

                free(dup);
            } else {
                runner = runner->next;
            }
        }
    }
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    struct Node *sorted = NULL, *unsorted = NULL;

    insert(&sorted, 10);
    insert(&sorted, 10);
    insert(&sorted, 20);
    insert(&sorted, 20);
    insert(&sorted, 30);

    insert(&unsorted, 30);
    insert(&unsorted, 10);
    insert(&unsorted, 20);
    insert(&unsorted, 10);
    insert(&unsorted, 30);

    printf("Sorted DLL before: ");
    display(sorted);
    removeSortedDuplicates(sorted);
    printf("Sorted DLL after:  ");
    display(sorted);

    printf("Unsorted DLL before: ");
    display(unsorted);
    removeUnsortedDuplicates(unsorted);
    printf("Unsorted DLL after:  ");
    display(unsorted);

    return 0;
}
```

Output:

```
Sorted DLL before: 10 10 20 20 30
Sorted DLL after:  10 20 30
Unsorted DLL before: 30 10 20 10 30
Unsorted DLL after:  30 10 20
```

8. Calculate the frequency count of a given integer in a circular doubly linked list.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

void insert(struct Node **head, int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = data;

    if (*head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        *head = newNode;
        return;
    }

    newNode->next = *head;
    newNode->prev = (*head)->prev;
    (*head)->prev->next = newNode;
    (*head)->prev = newNode;
}

int frequency(struct Node *head, int key) {
    int count = 0;
    struct Node *temp = head;

    if (head == NULL)
        return 0;

    do {
        if (temp->data == key)
            count++;

        temp = temp->next;
    } while (temp != head);

    return count;
}

int main() {
    struct Node *head = NULL;
    int key;

    insert(&head, 10);
    insert(&head, 20);
    insert(&head, 10);
    insert(&head, 30);
    insert(&head, 10);

    key = 10;

    printf("Frequency of %d = %d\n",
           key, frequency(head, key));

    return 0;
}
```

Output:

```
Frequency of 10 = 3
```

9. Search for a particular element in a binary tree.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

int search(struct Node *root, int key) {
    if (root == NULL)
        return 0;

    if (root->data == key)
        return 1;

    return search(root->left, key) ||
           search(root->right, key);
}

int main() {
    struct Node *root = createNode(10);

    root->left = createNode(20);
    root->right = createNode(30);
    root->left->left = createNode(40);
    root->left->right = createNode(50);

    if (search(root, 50))
        printf("Element 50 is found.\n");
    else
        printf("Element 50 is not found.\n");

    return 0;
}
```

Output:

```
Element 50 is found.
```

10. Calculate the height or depth of a tree and print it.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
```

```
    struct Node *right;
};

struct Node *createNode(int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

int height(struct Node *root) {
    int leftHeight, rightHeight;

    if (root == NULL)
        return 0;

    leftHeight = height(root->left);
    rightHeight = height(root->right);

    return (leftHeight > rightHeight ?
            leftHeight : rightHeight) + 1;
}

int main() {
    struct Node *root = createNode(1);

    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);
    root->left->left->left = createNode(6);

    printf("Height of the tree = %d\n", height(root));

    return 0;
}
```

Output:

```
Height of the tree = 4
```

11. Given a binary tree, convert it to a doubly linked list keeping the same order (inorder traversal).

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
};

struct ListNode {
    int data;
    struct ListNode *prev;
    struct ListNode *next;
};

struct TreeNode *createTreeNode(int data) {
    struct TreeNode *newNode =
        (struct TreeNode *)malloc(sizeof(struct TreeNode));
```

```
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

void convert(struct TreeNode *root,
             struct ListNode **head,
             struct ListNode **tail) {
    struct ListNode *newNode;

    if (root == NULL)
        return;

    convert(root->left, head, tail);

    newNode = (struct ListNode *)malloc(sizeof(struct ListNode));
    newNode->data = root->data;
    newNode->prev = *tail;
    newNode->next = NULL;

    if (*tail != NULL)
        (*tail)->next = newNode;
    else
        *head = newNode;

    *tail = newNode;

    convert(root->right, head, tail);
}

void display(struct ListNode *head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
}

int main() {
    struct TreeNode *root = createTreeNode(4);
    struct ListNode *head = NULL, *tail = NULL;

    root->left = createTreeNode(2);
    root->right = createTreeNode(6);
    root->left->left = createTreeNode(1);
    root->left->right = createTreeNode(3);
    root->right->left = createTreeNode(5);
    root->right->right = createTreeNode(7);

    convert(root, &head, &tail);

    printf("Doubly linked list: ");
    display(head);

    return 0;
}
```

Output:

```
Doubly linked list: 1 2 3 4 5 6 7
```

12. Given a binary tree, connect the nodes at the same level using a next-right pointer.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
```

```
    int data;
    struct Node *left;
    struct Node *right;
    struct Node *nextRight;
};

struct Node *createNode(int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    newNode->nextRight = NULL;

    return newNode;
}

void connect(struct Node *root) {
    struct Node *queue[50];
    int front = 0, rear = 0;
    int levelCount, i;

    if (root == NULL)
        return;

    queue[rear++] = root;

    while (front < rear) {
        levelCount = rear - front;

        for (i = 0; i < levelCount; i++) {
            struct Node *current = queue[front++];

            if (i < levelCount - 1)
                current->nextRight = queue[front];
            else
                current->nextRight = NULL;

            if (current->left != NULL)
                queue[rear++] = current->left;

            if (current->right != NULL)
                queue[rear++] = current->right;
        }
    }
}

void displayLevels(struct Node *root) {
    struct Node *queue[50];
    int front = 0, rear = 0;
    int levelCount, i;

    queue[rear++] = root;

    while (front < rear) {
        levelCount = rear - front;

        for (i = 0; i < levelCount; i++) {
            struct Node *current = queue[front++];

            printf("%d", current->data);

            if (current->nextRight != NULL)
                printf(" -> %d  ", current->nextRight->data);
            else
                printf(" -> NULL  ");

            if (current->left != NULL)
                queue[rear++] = current->left;

            if (current->right != NULL)
```

```
                queue[rear++] = current->right;
        }

        printf("\n");
    }
}

int main() {
    struct Node *root = createNode(1);

    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);
    root->right->right = createNode(6);

    connect(root);

    printf("Next-right connections:\n");
    displayLevels(root);

    return 0;
}
```

Output:

```
Next-right connections:
1 -> NULL
2 -> 3  3 -> NULL
4 -> 5  5 -> 6  6 -> NULL
```

13. Implement a priority queue using a binary search tree and perform insertion and deletion operations.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int priority;
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(int priority) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->priority = priority;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

struct Node *insert(struct Node *root, int priority) {
    if (root == NULL)
        return createNode(priority);

    if (priority < root->priority)
        root->left = insert(root->left, priority);
    else
        root->right = insert(root->right, priority);

    return root;
}

struct Node *deleteMax(struct Node *root, int *value) {
```

```
    struct Node *temp;

    if (root->right == NULL) {
        *value = root->priority;
        temp = root->left;
        free(root);
        return temp;
    }

    root->right = deleteMax(root->right, value);
    return root;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->priority);
        inorder(root->right);
    }
}

int main() {
    struct Node *root = NULL;
    int deleted;

    root = insert(root, 30);
    root = insert(root, 10);
    root = insert(root, 50);
    root = insert(root, 20);
    root = insert(root, 40);

    printf("Priority queue (low to high): ");
    inorder(root);

    root = deleteMax(root, &deleted);

    printf("\nDeleted highest priority: %d\n", deleted);

    printf("Priority queue after deletion: ");
    inorder(root);

    return 0;
}
```

Output:

```
Priority queue (low to high): 10 20 30 40 50
Deleted highest priority: 50
Priority queue after deletion: 10 20 30 40
```

14. In a given binary search tree, write a routine to delete a node with no child, one child and two children.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = data;
```

```
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

struct Node *insert(struct Node *root, int data) {
    if (root == NULL)
        return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);

    return root;
}

struct Node *minValueNode(struct Node *node) {
    struct Node *current = node;

    while (current->left != NULL)
        current = current->left;

    return current;
}

struct Node *deleteNode(struct Node *root, int key) {
    struct Node *temp;

    if (root == NULL)
        return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);
    else if (key > root->data)
        root->right = deleteNode(root->right, key);
    else {
        if (root->left == NULL) {
            temp = root->right;
            free(root);
            return temp;
        }

        if (root->right == NULL) {
            temp = root->left;
            free(root);
            return temp;
        }

        temp = minValueNode(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }

    return root;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main() {
    struct Node *root = NULL;

    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
```

```
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    printf("Original BST: ");
    inorder(root);

    root = deleteNode(root, 20);
    printf("\nAfter deleting leaf 20: ");
    inorder(root);

    root = deleteNode(root, 30);
    printf("\nAfter deleting node 30: ");
    inorder(root);

    root = deleteNode(root, 50);
    printf("\nAfter deleting node 50: ");
    inorder(root);

    return 0;
}
```

Output:

```
Original BST: 20 30 40 50 60 70 80
After deleting leaf 20: 30 40 50 60 70 80
After deleting node 30: 40 50 60 70 80
After deleting node 50: 40 60 70 80
```

15. Convert the given postfix expression into an expression tree.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

struct Node {
    char data;
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(char data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}

struct Node *stack[50];
int top = -1;

void push(struct Node *node) {
    stack[++top] = node;
}

struct Node *pop() {
    return stack[top--];
}

struct Node *buildTree(char postfix[]) {
    int i;
    struct Node *node, *left, *right;
```

```
    for (i = 0; postfix[i] != '\0'; i++) {
        if (isalnum(postfix[i])) {
            push(createNode(postfix[i]));
        } else {
            node = createNode(postfix[i]);
            right = pop();
            left = pop();

            node->left = left;
            node->right = right;

            push(node);
        }
    }

    return pop();
}

void inorder(struct Node *root) {
    if (root != NULL) {
        if (root->left != NULL)
            printf("(");

        inorder(root->left);
        printf("%c", root->data);
        inorder(root->right);

        if (root->right != NULL)
            printf(")");
    }
}

int main() {
    char postfix[] = "ab+c*";
    struct Node *root = buildTree(postfix);

    printf("Postfix expression: %s\n", postfix);
    printf("Infix expression: ");
    inorder(root);
    printf("\n");

    return 0;
}
```

Output:

```
Postfix expression: ab+c*
Infix expression: ((a+b)*c)
```

16. Construct a binary search tree and display its inorder, preorder and postorder traversals.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left;
    struct Node *right;
};

struct Node *createNode(int data) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
```

```
    return newNode;
}

struct Node *insert(struct Node *root, int data) {
    if (root == NULL)
        return createNode(data);

    if (data < root->data)
        root->left = insert(root->left, data);
    else
        root->right = insert(root->right, data);

    return root;
}

void inorder(struct Node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

void preorder(struct Node *root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct Node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    struct Node *root = NULL;
    int values[] = {50, 30, 70, 20, 40, 60, 80};
    int i;

    for (i = 0; i < 7; i++)
        root = insert(root, values[i]);

    printf("Inorder: ");
    inorder(root);

    printf("\nPreorder: ");
    preorder(root);

    printf("\nPostorder: ");
    postorder(root);

    return 0;
}
```

Output:

```
Inorder: 20 30 40 50 60 70 80
Preorder: 50 30 20 40 70 60 80
Postorder: 20 40 30 60 80 70 50
```

17. Implement heap sort for a set of numbers.

Program:

```
#include <stdio.h>

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    int temp;

    if (left < n && a[left] > a[largest])
        largest = left;

    if (right < n && a[right] > a[largest])
        largest = right;

    if (largest != i) {
        temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n) {
    int i, temp;

    for (i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);

    for (i = n - 1; i > 0; i--) {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

int main() {
    int a[] = {50, 20, 30, 25, 15, 22, 32};
    int n = 7;
    int i;

    printf("Before sorting: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    heapSort(a, n);

    printf("\nAfter heap sort: ");
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Output:

```
Before sorting: 50 20 30 25 15 22 32
After heap sort: 15 20 22 25 30 32 50
```